

HLD-as-Source-of-Intent: A Ten-Agent Closed-Loop System for Intent-Based Networking Grounded in RFC 9315

MARC BRUYÈRE-YAMADA, Télécom SudParis, France and JFLI — Japanese-French Laboratory for Informatics, Japan
ANONYMOUS CO-AUTHORS, TBD,

We present an Intent-Based Networking (IBN) closed-loop **control** system in which the operator’s High-Level Design (HLD) document, stored as Markdown under version control, is the sole source **of intent**. **A git commit on the HLD is the only external trigger: the system reconciles live network state to the committed design through a pipeline of ten specialised agents mapped to the stages defined in RFC 9315**, backed by a Neo4j graph Single Source of Truth (SSoT) with nine ontology layers and five concurrent model states (What-If, Candidate, Plan-of-Record, Deployed, As-Built). A three-tier memory architecture separates structured state (Neo4j), working memory (Live-Memory, MCP-attached), and long-term knowledge (Graph-Memory with BGE-M3 vector embeddings and ontology-guided entity extraction).

We describe the two-loop control decomposition (autonomous inner loop, human-in-the-loop outer loop), the nine-layer knowledge graph ontology with its six cross-layer relationship types, the ten agents with their precise inputs, outputs, dispatch tables, and write-layer assignments, and the three-tier memory with its LLM-driven consolidation pipeline. We report an end-to-end implementation covering multi-vendor rendering across VyOS, Nokia SR Linux, and FRRouting, shipped through six incremental “slices” on a live Containerlab testbed with 16 nodes on a 12 GB ARM64 host. Configuration is pushed through both CLI-over-docker `exec` and schema-validated gNMI paths, with the same A3-rendered intent artefact feeding either transport. The core pipeline is deterministic and LLM-free; large language models are used exclusively for memory consolidation and knowledge extraction, keeping the fulfilment path’s cycle time under 10 seconds and its correctness properties non-probabilistic.

Additional Key Words and Phrases: intent-based networking, RFC 9315, closed-loop control, GitOps, knowledge graphs, multi-agent systems, gNMI, YANG, network automation, Neo4j

1 INTRODUCTION

1.1 The HLD–Production Gap

Modern enterprise and campus networks continue to be operated through an uncomfortable split: architects produce dense High-Level Design (HLD) documents that capture the *intent* of the network—its sites, zones, VLAN plans, firewall policies, routing design—**while operators, at a much later stage, translate those documents by hand into device-specific configuration**. The HLD is authoritative on paper but inert in production: it does not execute, it does not verify itself against the live topology, and it inevitably drifts from the devices it is meant to describe.

This split has several well-documented consequences. First, the HLD becomes a stale artefact within weeks of initial deployment: operators make out-of-band changes to address urgent issues, and the document is never backfilled. Second, the translation from HLD to device configuration is manual, error-prone, and vendor-specific;

an architect’s VLAN plan expressed in a prose table must be separately re-encoded as VyOS `set` commands, Nokia SR Linux `set /` statements, or FRR `vytysh` directives. Third, the *rationale* for a design choice—captured naturally in prose and margin notes in the HLD—is lost the moment the intent is translated into a structured policy form. When a firewall rule is later found to be overly permissive, the operator who must decide whether to narrow it has no machine-accessible record of *why* the rule was written that way.

1.2 Intent-Based Networking and RFC 9315

Closing this gap is the central premise of Intent-Based Networking (IBN), formalised by the IETF in RFC 9315 [4]. The RFC defines intent as an abstract, high-level policy that the network infrastructure itself is responsible for rendering, deploying, and continuously verifying through two nested feedback loops. The *fulfilment* phase ingests intent, decomposes it into vendor-specific configuration, and orchestrates its deployment. The *assurance* phase monitors the deployed state, assesses compliance against the intent-derived Plan-of-Record, and either auto-remediates or escalates to a human operator. RFC 9315 further distinguishes the *inner loop* (autonomous, machine-to-machine, seconds-to-minutes cadence) from the *outer loop* (human-in-the-loop, minutes-to-days cadence).

Despite a decade of vendor activity around IBN—Cisco DNA Center, Juniper Apstra, Arista CloudVision, Nokia NSP—most deployed systems remain tightly coupled to a single vendor’s management plane and treat intent as a GUI-authored artefact inside a controller. Operators end up writing intent twice: once in the HLD (for humans) and once in the controller (for machines). The two representations drift, and the controller’s intent model is a proprietary abstraction that cannot be diffed, branched, or code-reviewed with standard development tooling.

1.3 Contribution: HLD-as-Intent

This paper describes a research prototype, developed on an open Containerlab testbed [1], that treats the HLD Markdown document itself as the source of intent. A `git commit` on the HLD is the only external trigger the system accepts: everything downstream—parsing the change, updating the graph-structured Single Source of Truth (SSoT), decomposing intent into policy, rendering vendor-specific configuration, planning the migration with blast-radius analysis, pushing to the devices, and verifying the outcome against the committed design—is carried out autonomously by a pipeline of ten specialised agents whose roles map directly onto the stages of the RFC 9315 control loop. Architects edit the document they already know how to write; the loop handles the rest.

The contributions of this paper are fivefold:

Authors’ addresses: Marc Bruyère-Yamada, Télécom SudParis, Évry-Courcouronnes, France, JFLI — Japanese-French Laboratory for Informatics, Tokyo, Japan, marc@bruyere-yamada.name; Anonymous Co-authors, TBD, .

- C1. HLD-as-Intent.** We demonstrate that a conventional Markdown HLD, augmented only with a structured Device Inventory Table and a VLAN assignment table, is sufficient to drive a full IBN fulfilment-and-assurance loop end-to-end, with `git` providing provenance, diffing, branching, and human review for free. The HLD is not a wrapper around a proprietary intent model; it is the intent, and the system parses it directly **using a 584-line Markdown parser** with no LLM in the parsing path.
- C2. A ten-agent decomposition grounded in RFC 9315.** We map the abstract stages of RFC 9315 §5 onto ten concrete agents (A1–A10) with narrow, testable responsibilities. Each agent writes to a well-defined subset of a nine-layer graph ontology (3,969 lines of Python across the ten agents), each emits structured working-memory notes that can be consolidated into long-term knowledge, and each publishes typed events to a Redis Streams event bus. The decomposition is fine-grained enough that adding a new vendor requires exactly one line per dispatch table plus a Jinja2 template chain—no agent code changes.
- C3. A nine-layer knowledge-graph ontology.** The Neo4j SSOT organises network state across nine layers (L1 Infrastructure through L9 People/Process) connected by six typed cross-layer relationships (REALIZED_BY, ABSTRACTS, AGGREGATES, CONTAINS, TRAVERSES, PRECEDED_BY). Five concurrent model states (WHAT_IF, CANDIDATE, POR, DEPLOYED, AS_BUILT) co-exist as overlapping subgraphs, enforced by a controller with eight legal transition edges and four rollback paths. The ontology is codified in 616 lines of Cypher DDL with profile-based constraint bundles.
- C4. A three-tier memory architecture.** Beyond the Neo4j SSOT (Tier 1: *what is*) we layer a Live-Memory tier (Tier 2: *why it is so*—working memory for agent reasoning, exposed via MCP Streamable HTTP) and a Graph-Memory tier (Tier 3: *what was learned*—long-term knowledge graph with BGE-M3 1024-dimension embeddings in Qdrant, ontology-guided entity extraction across 14 entity classes and 8 relationship types). The three tiers are decoupled: the pipeline writes only to Tiers 1 and 2; consolidation from Tier 2 to Tier 3 runs asynchronously and is the only point where an LLM is invoked.
- C5. An incrementally shipped multi-vendor implementation.** We report on six “slices” of shipped functionality, culminating in a schema-validated gNMI push path for Nokia SR Linux running side-by-side with CLI-over-docker-exec paths for VyOS and FRRouting [2]. The testbed runs 16 Containerlab nodes (3 SR Linux, 2 FRR, 4 VyOS, 7 host stubs) on a single 12 GB ARM64 host with a full-cycle time of 8–10 s. The implementation totals 140 unit tests across 23 test suites with zero regressions across slices.

1.4 Paper Organisation

The remainder of this paper is organised as follows. Section 2 describes the closed-loop architecture and its grounding in RFC 9315, including the trigger-to-push pipeline, the approval gate, and the

Table 1. Mapping of RFC 9315 stages to the ten agents.

RFC 9315	Phase	Agent	Name
§5.1.1	Fulfilment	A1	Ingestion
§5.1.2	Fulfilment	A2	Intent→Policy
§5.1.2	Fulfilment	A3	Policy→Config
§5.1.2	Fulfilment	A4	Planning
§5.1.3	Fulfilment	A5	Orchestration
§5.2.1	Assurance	A6	Monitoring
§5.2.2	Assurance	A7	Assessment
§5.2.3	Assurance	A8	Action
§5.2	Assurance	A9	Abstraction
§5.2	Assurance	A10	Reporting

invariants that make the system **introspectable**. Section 3 describes the Neo4j nine-layer ontology, the six cross-layer relationship types, the five concurrent model states, and the profile-based schema governance. Section 4 details each of the ten agents, covering their inputs, outputs, Neo4j write targets, dispatch tables, and key design decisions. Section 5 covers the three-tier memory architecture, the consolidation pipeline, the IBN Lifecycle Ontology, and the graph-guided RAG query path. Section 6 reports on the incremental implementation across six slices, the testbed configuration, measurements, and failure modes encountered and resolved. Section 7 discusses related work and lessons learned. Section 8 concludes.

2 THE CLOSED LOOP

2.1 Grounding in RFC 9315

RFC 9315 [4] decomposes an IBN system into two nested feedback loops. The *inner loop* is autonomous: it runs on the order of seconds to minutes, compares the network’s observed operational state against the intent-derived Plan-of-Record, detects drift, classifies severity, and either self-corrects (re-render, re-push) or escalates. The *outer loop* is on human timescales: operators review reports, amend the intent, and re-submit; the system adapts accordingly. RFC 9315 further partitions the work into *fulfilment* (§5.1: intent ingestion, policy translation, config rendering, orchestration) and *assurance* (§5.2: monitoring, assessment, remediation, abstraction, reporting).

Our system retains this two-loop, two-phase decomposition verbatim. Table 1 shows the mapping between RFC 9315 subsections and our ten agents. Beyond the standard decomposition, we make three additional architectural commitments:

AC1. A single external trigger. The only event that enters the system from outside is `git commit` on the HLD repository. A post-commit hook invokes the pipeline entry point `HldCommitPipeline.run(commit, hld_path)`, which is the sole supported way to start a fulfilment cycle. The hook runs the pipeline in a background process, emits a banner to the terminal (`[ibn-hook] HLD changed in <sha>`), and deposits a pending-approval marker when the approval gate fires. Everything else is internal. There is no GUI, no REST API, no controller; the intent surface is the text editor and the `git` CLI.

AC2. A single human checkpoint. The inner loop is autonomous by design, but change-of-intent operations (new devices, decommissions, policy scope changes) are gated by an explicit approval step between Agent 4 (Planning) and Agent 5 (Orchestration). The approval artefact is a Markdown file deposited under `.git/ibn-pending-approval/<sha>.md` containing the severity classification, blast radius, affected devices, and a forward/inverse summary of the planned change. An operator approves by running `python -m ibn.tools.approve <sha>` or rejects with `python -m ibn.tools.approve reject <sha> --reason "..."`. This preserves RFC 9315's outer-loop human oversight but removes the GUI-controller dependency.

AC3. Deterministic pipeline, LLM-assisted memory. The core pipeline (A1→A5 fulfilment, A6→A8 assurance) is entirely LLM-free. LLMs enter the architecture exclusively at the memory consolidation and knowledge extraction boundary (Section 5), which runs asynchronously after cycle completion. This separation is intentional: the pipeline's correctness properties—idempotent renders, atomic gNMI commits, rollback chains via PRECEDED_BY—depend on determinism. The memory tier's value proposition—turning thousands of structured agent notes into queryable operational knowledge—is precisely where LLM summarisation excels. The architecture puts each capability where it pays off.

2.2 Trigger-to-Push Pipeline

When a `git commit` touches the HLD file (`Enterprise_Campus_Network_HLD.md`), the post-commit hook spawns `HldCommitPipeline.run(commit_sha, hld_path)`. The pipeline carries out twelve stages in order; each stage is described below with its agent, inputs, outputs, and the Neo4j layer it writes to.

Stage 1. Diff (parser). The HLD Markdown is parsed by a deterministic 584-line parser (`hld_parser.py`) that extracts structured sections: the Device Inventory Table, population VLAN tables, DMZ VLAN tables, and firewall policy matrices. The parser produces a typed `HldChangeset` dataclass with fields: `populations_added`, `populations_removed`, `populations_modified`, `dmz_added`, `dmz_removed`, `dmz_modified`, `devices_added`, `devices_removed`, `devices_modified`. Each field is a list of domain-specific typed records (e.g. `HldDevice` with fields `hostname`, `vendor`, `platform`, `role`, `mgmt_ipv4`). The changeset is computed by diffing the parsed state of the new commit against the previous commit's parsed state.

Stage 2. Agent 1 (Ingestion). The ingestion agent (`agent1_ingestion.py`, 584 lines) receives the changeset and upserts nodes into Neo4j at layers L1 (Device, VLAN, Site) and L4 (Intent). Every node is created with `modelState = CANDIDATE`. Idempotency is enforced by keying each intent on its HLD origin: `HLD:<path>:<line_number>`. If an intent with the same origin already exists, the agent updates rather than duplicates. Conflict detection queries all active intents and checks for `CONFLICTING_ACTION` (same target,

incompatible action) or `DUPLICATE` (same target, same action). Each write is dual-sinked: a `live_note()` call to the `ibn-candidate-<version>` Live-Memory space records the write's context (what was written, what was considered, why). Events are published to the Redis Streams event bus: `intent.ingested`, `device.added`, `device.modified`, `device.removed`.

For device changes, Agent 1 also handles lifecycle state: new devices enter as `lifecycleState = PLANNED`, modified devices are updated in place, and removed devices are marked `RETIRED` (soft delete—the node stays in the graph for audit).

Stage 3. Provisioning. For any device deltas, the Provisioner (`provisioner.py`) mutates the Containerlab topology YAML and triggers a `clab deploy --reconfigure`. The Provisioner has two code paths:

- `provision(device_entry)`: single-device path. Adds the node to the YAML, runs `clab deploy --reconfigure`, waits for the container, records a `LifecycleEvent` at L8.
- `provision_batch(device_entries)`: multi-device path (used when `|devices_added| > 1`). Performs a single YAML write and a single `clab deploy --reconfigure` for the entire batch. This was a mid-Slice-5 fix: the per-device path ran `clab deploy --reconfigure` once per device, and since `--reconfigure` re-processes the entire topology each time, four sequential deploys exceeded the 300 s timeout on the third iteration. The batch path completes in ~10 s.

The vendor→containerlab-kind mapping is driven by a dispatch table `_KIND_MAP`:

Listing 1. Vendor dispatch for Containerlab kinds.

```
1 _KIND_MAP = {
2     ("nokia", "srlinux"): {"kind": "nokia_srlinux",
3                             "image": "ghcr.io/nokia/srlinux:latest",
4                             "type": "ixrd21"},
5     ("vyos", "vyos"): {"kind": "linux",
6                         "image": "ghcr.io/sysoleg/vyos-container:latest"},
7     ("frr", "frr"): {"kind": "linux",
8                      "image": "ibn-frr:local"},
9 }
```

Adding a new vendor to the provisioning path is one entry in this table plus a Dockerfile for the container image.

Stage 4. Agent 2 (Intent→Policy). The policy decomposition agent (`agent2_intent_policy.py`, 221 lines) receives the list of intent IDs produced by Stage 2 and decomposes each into Policy and FirewallRule nodes at L4. Policy IDs follow the pattern `POL-<intent_id>`; rule IDs follow `RUL-<intent_id>-<idx:02d>`. The decomposition is driven by an `ACCESS_MATRIX` dispatch table that maps a target string (e.g. "Internet+DMZ", "DNS-DMZ") to a list of concrete firewall rules with fields `destZone`, `action`, `protocol`, `destPort`, and `description`.

Zone inference is currently rule-based: DMZ targets map to `DMZ_INFRA`, guest/contractor/visitor targets to `GUEST`, all others to `USER`. Device-only commits (no population or DMZ changes) short-circuit this stage entirely: Agent 2 reports skipped and the pipeline continues with the existing intent set.

Stage 5. Agent 4 (Planning). The planning agent (agent4_planning.py, 243 lines) produces a MigrationPlan node at L7 with:

- A **severity classification** computed by a deterministic heuristic: HIGH if devices were added or removed; MEDIUM if populations or DMZ segments were removed; LOW if populations or DMZ segments were added or modified; INFO if no changes detected.
- A **blast radius**: the count and list of affected devices, computed by traversing Configuration→Device edges (APPLIES_TO).
- A **forward summary** (what will change) and an **inverse summary** (how to undo it), both in human-readable prose.

Plan IDs follow the pattern PLAN-<commit_sha[:8]>. If a prior PENDING plan exists for the same intent set, it is automatically marked SUPERSEDED before the new plan is written.

Stage 6. Approval gate. If severity is HIGH (device additions or removals), the pipeline deposits a Markdown summary under .git/ibn-pending-approval/<sha>.md and halts. The summary contains: severity, blast radius, affected device list, forward and inverse summaries, and the command to approve. The operator reviews and runs:

```
python -m ibn.tools.approve <sha>
```

The approve tool (ibn.tools.approve) transitions the MigrationPlan from PENDING to APPROVED, records the operator identity and timestamp, cleans up the pending Markdown, and invokes HldCommitPipeline.resume_from_approval(plan_id).

For LOW/INFO severity, the gate auto-approves (configurable via IBN_AUTO_APPROVE=1 for test environments).

Stage 7. Agent 3 (Policy→Config). The config rendering agent (agent3_policy_config.py, 420 lines) is the most data-intensive stage. It executes ten named Cypher queries against Neo4j to build a rendering context: firewalls (FirewallPair→Device with modelState IN ['POR', 'DEPLOYED']), zones (ordered by securityLevel), segments (Segment→VLAN via MAPPED_TO_VLAN), policies (FirewallRule with legacy field coalescing), intents, fw_interfaces, nat_rules, fw_transit (VLAN 300 with subnets), switches (non-firewall devices excluding RETIRED), and vlans.

For each device, Agent 3 resolves the appropriate Jinja2 template chain from the _TEMPLATE_CHAINS dispatch table:

Listing 2. Template chains per vendor.

```
1 _TEMPLATE_CHAINS = {
2   "vyos": ["vyos_base.j2", "vyos_policies.j2",
3           "vyos_nat.j2", "vyos_ha.j2"],
4   "srlinux": ["sr1_base.j2", "sr1_interfaces.j2",
5              "sr1_vlans.j2"],
6   "frr": ["frr_base.j2", "frr_interfaces.j2",
7           "frr_routing.j2"],
8 }
```

Each template in the chain is rendered against the Neo4j context and concatenated. The resulting configuration blob

is written as a Configuration node at L5 with a content hash (SHA-256[:16]) for change detection. A diff against the previous configuration for the same device is computed to determine whether the render produced net-new content.

Stage 8. Agent 5 (Orchestration). The orchestration agent (agent5_orchestration.py, ~350 lines) pushes each rendered configuration to its live device. The executor is resolved from a two-level dispatch:

Listing 3. Executor dispatch with transport override.

```
1 _VENDOR_EXECUTORS = {
2   "vyos": _default_ssh_executor,
3   "srlinux": _srlinux_ssh_executor,
4   "frr": _frr_executor,
5 }
6
7 def _resolve_executor(platform):
8   if platform == "srlinux" and
9     os.environ.get("IBN_SRLINUX_TRANSPORT")
10    == "gnmi":
11     return _srlinux_gnmi_executor
12   return _VENDOR_EXECUTORS.get(platform,
13     _default_ssh_executor)
```

Four executor implementations exist:

VyOS: paramiko SSH to the device, pipe configuration commands.

SR Linux (CLI): docker exec -i <container> sr_cli, wrapping commands in enter candidate / ... / commit now / quit.

SR Linux (gNMI): A narrow CLI→YANG path translator (_srlinux_yang.py, ~120 lines) converts the rendered set / commands into gNMI (path, value) tuples. The executor issues a single SetRequest via pygnmi [8] to <mgt-ipv4>:57400 with TLS (self-signed cert, skip_verify=True). The translator has ten rules covering the exact line shapes the three sr1_*.j2 templates emit; unknown lines raise UnknownCliLine to prevent silent config loss. Multiple leaves under the same container path are merged into a single gNMI update, reducing wire operations from 10 to 7 for a typical render.

FRR: docker exec -i <container> vtysh, wrapping in enable / configure terminal / ... / end / write memory.

Each successful push records a DeploymentEvent node at L5. Devices with empty rendered content (e.g. device-only commits where templates produce no new output) are skipped.

Stage 9. Agent 6 (Monitoring). The monitoring agent (agent6_monitoring.py) collects post-deployment telemetry via CLI polling (show commands over SSH or docker exec). For each device it creates Telemetry nodes and upserts OperationalState nodes at L5 with modelState = AS_BUILT. Metrics include interface state, VRRP role, BGP peer status, and OSPF neighbour count. The monitoring interval is configurable (default 30 s for the inner loop).

Stage 10. Agent 7 (Assessment). The assessment agent (agent7_assessment.py, 417 lines) compares the POR and AS_BUILT subgraphs. It fetches per-intent POR configurations and

per-device As-Built state, then runs three registered drift detectors:

firewall_rule: Detects missing or extra rules by comparing POR FirewallRule sets against As-Built running configuration snippets.

interface_state: Detects interface admin/oper-state mismatches.

vrp_role: Detects VRRP role mismatches (primary/backup).

Each drift produces a ComplianceAssessment node at L6 with a verdict (COMPLIANT, DEGRADED, NON_COMPLIANT, UNKNOWN) and a compliance percentage. For non-compliant verdicts, a root-cause tracer traverses the dependency graph (via REALIZED_BY/AGGREGATES edges) to identify the causal chain and assigns a confidence level (HIGH or LOW).

If drift events are detected, an Incident node is created at L6, linked to the assessment. Severity classification: CRITICAL (≥ 3 drift events), HIGH (non-compliant verdict), MEDIUM (default).

Stage 11. Agent 8 (Action). The action agent (agent8_action.py, 405 lines) receives incident data from Agent 7 and applies a five-level autonomy decision matrix:

Table 2. Agent 8 autonomy decision matrix.

Severity	Action	Auto	Human
INFO	Log only	✓	
LOW	Auto-remediate	✓	
MEDIUM	Remediate + verify	✓	
HIGH	Escalate		✓
CRITICAL	Rollback + escalate		✓

Auto-remediation re-invokes A3→A5 on the narrowed artefact set for the affected devices. Rollback walks the PRECEDED_BY chain one step to retrieve the previous Configuration node and re-pushes it. Every action is recorded as a Remediation node at L6 with fields: remediationId, intentId, action, success, error, escalated, rollbackCount.

Stage 12. Agents 9–10 (Abstraction, Reporting). Agent 9 aggregates device-level observations into intent-level compliance scores using ABSTRACTS edge traversals. Agent 10 generates human-facing compliance reports. Both are designed but not yet fully implemented in the current prototype; their stubs emit aggregate metrics to the event bus and write summary data to the graph.

2.3 Pipeline Timing

A typical no-op HLD edit (comment change, no structural delta) completes the full twelve-stage cycle in 8–10 s against the current testbed (16 Containerlab nodes on a 12 GB ARM64 host). The dominant costs are Neo4j Bolt round-trips (~200 ms per Cypher query, ten queries in A3) and container-exec I/O (~1 s per device push). A device-addition commit, which goes through the approval gate

and a clab deploy --reconfigure, completes in 20–40 s of wall-clock time once the human approval is granted. The approval itself is out-of-band and untimed.

2.4 Invariants

Three invariants hold across every stage of the pipeline and are load-bearing for the system’s correctness:

I1: Every agent write is dual-sink. Every node or edge an agent writes to Neo4j is accompanied by a structured live_note() call into a Live-Memory space keyed on the current model state. This preserves the *why* of the write (prompt, inputs considered, alternatives rejected) alongside the *what* (the graph mutation itself). Invariant I1 is enforced by the BaseAgent class: every agent inherits from BaseAgent, which provides a _note() helper that writes to both sinks atomically. An agent that calls neo4j.create_* without a corresponding _note() violates the contract; this is detectable by static analysis (grep for unmatched writes).

I2: Every node carries modelState. The five-valued modelState property (WHAT_IF, CANDIDATE, POR, DEPLOYED, AS_BUILT) is written on every node creation and transitioned through well-defined edges. A legal transition table (eight edges, four rollback paths) is enforced by the ModelStateController; illegal transitions raise ModelStateTransitionError at write time rather than corrupting the graph silently. Every transition creates a LifecycleTransition node at L8 linked to the affected intent via a HAS_TRANSITION edge.

I3: Every agent execution is recorded. Each agent run produces an AgentExecution node at L9 with the agent ID, start/end timestamps, inputs summary, outputs summary, success/failure status, and the intent ID being processed. This provides full traceability from intent to device to compliance assessment and back.

Together, I1–I3 ensure the system is fully introspectable: at any point, an operator can ask “what is POR right now?” (query modelState = ‘POR’ in Neo4j), “what changed in the last cycle?” (query recent AgentExecution and LifecycleTransition nodes), and “why did the agent do this?” (read the corresponding Live-Memory note) and get machine-structured answers.

3 SSOT: NINE-LAYER KNOWLEDGE-GRAPH ONTOLOGY

The Single Source of Truth (SSoT) is a Neo4j [3] property graph whose schema is organised as a *layered ontology*. The ontology is codified in 616 lines of Cypher DDL (Neo4j_Schema_Cypher.cypher) with constraints, indexes, and seed-data templates. Layering is load-bearing: each agent in the pipeline of Section 4 writes into a small, well-defined subset of layers, and every cross-layer relationship has an explicit, audit-friendly semantics. The ontology is inspired by Google’s MALT [9] multi-level topology abstraction, but specialised for the IBN lifecycle rather than for datacentre traffic engineering, and aligned with the fulfilment/assurance split of RFC 9315 [4].

3.1 The Nine Layers

Table 3 lists the nine layers with their canonical node labels. Labels are reserved tokens: every node in the graph carries exactly one *primary* layer label (e.g. Device, Intent, MigrationPlan) and may carry additional tag labels (e.g. :Active, :POR).

The layering is strictly semantic, not a storage partition: Neo4j holds all labels in a single graph so that queries can traverse across layers in a single Cypher statement. The value of the layer is that it constrains *who writes what*: Agent 1 only writes L1/L4; Agent 3 only writes L5; Agent 7 only writes L6. This makes each agent’s blast radius on the graph bounded and testable. A table of agents and their write layers is included in Table 3.

Layer design rationale. The nine layers correspond to nine qualitatively different concerns in the network lifecycle. Layers L1–L3 capture the static structure (infrastructure, topology, architecture) that changes on the timescale of HLD revisions. L4 captures intent and policy, which change on the timescale of business requirements. L5 captures the dynamic configuration and operational state that changes on the timescale of pipeline cycles. L6 captures incidents and compliance, which are generated as side effects of assurance. L7 captures planned changes, which exist between intent and deployment. L8 captures lifecycle transitions, which are the audit trail of state evolution. L9 captures the human and agent actors involved.

3.2 Cross-Layer Relationships

Six cross-layer relationship types form the backbone of the ontology. They are the *only* edges permitted to cross a layer boundary; intra-layer edges (e.g. CONNECTS_TO between two Interface nodes, PEER_OF between two Device nodes) are free-form within a layer.

REALIZED_BY (descending realisation). An Intent (L4) is REALIZED_BY a set of Policy/FirewallRule nodes (L4), which are in turn REALIZED_BY Configuration nodes (L5), which are REALIZED_BY the live OperationalState (L5). The REALIZED_BY chain is the primary audit trail: following it downward from an intent answers “does this intent actually exist on the wire?”. Agent 7 performs exactly this traversal during drift detection: it fetches the POR configuration for an intent via REALIZED_BY edges and compares it against the AS_BUILT operational state.

ABSTRACTS (ascending abstraction). The inverse of REALIZED_BY. Agent 9 uses ABSTRACTS edges to roll up device-level observations into intent-level compliance scores and then into site-level summaries.

AGGREGATES (set membership). Distinct from realisation. A FirewallPair (L2) AGGREGATES two Device nodes (L1); a Segment (L2) AGGREGATES a set of VLAN nodes (L1). AGGREGATES is structural—it says “these nodes form a group”—while REALIZED_BY is functional—it says “this node is the concrete form of that abstract node”.

CONTAINS (structural nesting). A Site (L1) CONTAINS Device nodes; an Architecture (L3) CONTAINS ArchitectureVersion nodes; an HLD (L3) CONTAINS Intent nodes extracted from

its sections. CONTAINS always points from a parent to its children within a single ownership hierarchy.

TRAVERSES (policy crossing). A FirewallRule (L4) TRAVERSES a FirewallPair (L2). An Intent (L4) TRAVERSES a Zone (L2) when its scope is zone-level. Blast-radius analysis in Agent 4 is largely a TRAVERSES traversal: “how many zones does this intent cross? how many firewall pairs does this rule set touch?”.

PRECEDED_BY (temporal ordering). Every Configuration node carries a PRECEDED_BY edge to the configuration it replaced. Rollback in Agent 8 is implemented by walking PRECEDED_BY one step and re-pushing the predecessor. The chain is unbounded; an operator can walk it back to the initial configuration to see the full history of a device’s config evolution.

3.3 Five Concurrent Model States

Every node in the graph carries a `modelState` property drawn from the set $S = \{\text{WHAT_IF}, \text{CANDIDATE}, \text{POR}, \text{DEPLOYED}, \text{AS_BUILT}\}$. The five states co-exist *simultaneously* as overlapping subgraphs over the same node labels and relationship types; selecting a state is a Cypher filter on `modelState`. This design is directly inspired by MALT’s [9] observation that multiple levels of abstraction must be maintained simultaneously for a topology to be operationally useful.

WHAT_IF. Exploratory scratch space used by Agent 4 during planning. Blast-radius simulations clone the affected subgraph into WHAT_IF state, mutate it, and discard it after analysis. The production graph is never touched.

CANDIDATE. State produced by a freshly committed HLD that has not yet been approved. Agent 1 writes new nodes directly as CANDIDATE. The CANDIDATE subgraph is the “proposed future”: it co-exists with the current POR until the approval gate promotes it.

POR (Plan-of-Record). The canonical intent: what the network *should* be. Transition from CANDIDATE to POR happens only after the approval gate is cleared. All rendering queries in Agent 3 filter for `modelState IN [‘POR’, ‘DEPLOYED’]` to ensure they operate against approved state.

DEPLOYED. Reflects what Agent 5 actually pushed, device by device. A Configuration node becomes DEPLOYED when the executor returns success. DEPLOYED may temporarily differ from POR during a rolling deployment when some devices have been updated and others have not.

AS_BUILT. The ground-truth view built from Agent 6’s telemetry. May diverge from DEPLOYED in the presence of out-of-band changes, partial push failures, or device-side configuration mutations. Agent 7’s drift detection is precisely a POR vs. AS_BUILT set-difference over the subgraph.

Legal transitions. The `ModelStateController` (298 lines) enforces eight legal transitions stored as a frozen set of (from, to) pairs:

Forward: WHAT_IF → CANDIDATE → POR → DEPLOYED → AS_BUILT
Rollback: CANDIDATE → WHAT_IF, POR → CANDIDATE, DEPLOYED → POR, AS_BUILT → DEPLOYED

Table 3. Nine-layer ontology: one canonical label set per layer, with the agent(s) that write to each layer.

Layer	Name	Representative labels	Writers
L1	Infrastructure	Site, Device, Interface, PhysicalLink, LogicalLink, VLAN, VRF, Subnet, IPAddress	A1
L2	Topology	Topology, TopologyLayer, Zone, Segment, FirewallPair, StackGroup	A1 (seed)
L3	Architecture	Architecture, ArchitectureVersion, HLD, ARD, BusinessUseCase	A1 (seed)
L4	Policy & Intent	Intent, Policy, FirewallRule, SGT, QoSPolicy	A1, A2
L5	Config & State	Configuration, DeploymentEvent, Telemetry, OperationalState	A3, A5, A6
L6	Incidents	Alert, Incident, RootCause, ComplianceAssessment, Remediation	A7, A8
L7	Change Mgmt	ChangeRequest, MigrationPlan	A4
L8	Lifecycle	LifecyclePhase, LifecycleTransition, LifecycleEvent	Provisioner, MSC
L9	People/Process	Operator, Team, Ticket, SLA, AgentExecution	All agents

Any other transition raises `ModelStateTransitionError`. Every transition creates a `LifecycleTransition` node at L8 with fields: `transitionId`, `intentId`, `fromState`, `toState`, `reason`, `operator`, `timestamp`. The controller also assigns each transition to a Live-Memory space via a prefix map:

Listing 4. Live-Memory space prefix per model state.

```

1 _SPACE_PREFIXES = {
2   "WHAT_IF": "ibn-whatif",
3   "CANDIDATE": "ibn-candidate",
4   "POR": "ibn-por",
5   "DEPLOYED": "ibn-deploy",
6   "AS_BUILT": "ibn-asbuilt",
7 }

```

Convenience methods `approve()`, `deploy()`, `verify()`, and `rollback()` wrap the generic `transition()` call with the appropriate (`from`, `to`) pair and a default reason string.

3.4 Profiles and Schema Governance

Neo4j constraints encode the ontology machine-checkably. Beyond unique-key constraints on node identifiers (e.g. `CREATE CONSTRAINT FOR (d:Device) REQUIRE d.deviceId IS UNIQUE`), the schema declares *profiles*: named bundles of Cypher constraints that capture deployment-specific invariants. Three profiles are defined:

Campus profile. Every Device must be `CONTAINS`-linked to exactly one Site. Every FirewallRule must `TRAVERSES` exactly one FirewallPair. Every VLAN must have a non-null `vlanId`. This profile matches the reference HLD topology.

Branch profile. Relaxes the Campus profile to allow devices without a FirewallPair (branch sites may use a cloud firewall), and allows VLANs without a local Segment assignment.

Lab profile. The most permissive profile. Allows devices without site assignment (for ad-hoc containerlab testing) and relaxes firewall-pair constraints.

Violating a profile constraint is a deployment-time Cypher error, not a runtime surprise. The active profile is set per environment via the `IBN_SCHEMA_PROFILE` variable.

Schema evolution is versioned semantically (`MAJOR.MINOR.PATCH`) with an explicit `OntologyVersion` node tracking which version is active. A six-month stability window on the MAJOR version gives agents a contract they can rely on, and a named stored-procedure

library over canonical Cypher queries insulates agents from MINOR-version column renames.

3.5 Why Nine Layers, Not Fewer

Early iterations of the ontology collapsed L7 (Change Management) into L8 (Lifecycle) and L6 (Incidents) into L5 (Config & State). Both collapses damaged query ergonomics and introduced write-lock contention:

- Separating L6 from L5 lets the assurance agents (A7, A8) own their own layer without competing for write-locks with the fulfilment agents (A3, A5, A6) operating on L5. In a concurrent inner loop where A6 is writing telemetry to L5 while A7 is writing assessments, the two agents touch disjoint label sets.
- Separating L7 from L8 decouples *planned* change (MigrationPlan, ChangeRequest) from *actual* transitions (LifecycleEvent, LifecyclePhase). This matters because the former are reasoned about *prospectively* (Agent 4 creates them before approval) and the latter are reasoned about *retrospectively* (the audit trail after the fact).
- Separating L9 from L7/L8 gives the agent execution audit trail its own namespace. Every agent writes an AgentExecution node to L9 on every run; mixing these high-frequency writes with the lower-frequency MigrationPlan nodes of L7 would pollute that layer's query results.

The nine-layer decomposition is the smallest we found that keeps each agent's write set disjoint across the most common concurrent execution patterns.

4 THE TEN AGENTS

Table 4 summarises the ten agents. Each agent inherits from a common `BaseAgent` class (181 lines) that provides: Neo4j and Live-Memory client injection, a `_note()` dual-sink helper (Invariant I1), an AgentExecution recording wrapper (Invariant I3), and a typed `_execute(**kwargs) -> dict` contract that each agent overrides.

The total implementation of the eight active agents (A1–A8) is 3,020 lines of Python. Adding A9 and A10 (currently stubs) will bring the total to approximately 3,500 lines.

Table 4. The ten agents: line counts, primary inputs, outputs, and write layers. Line counts are for the agent module only, excluding shared infrastructure.

Agent	Name	Lines	RFC	Input	Output
A1	Ingestion	584	\$5.1.1	HldChangeset	Intent, Device, VLAN
A2	Intent→Policy	221	\$5.1.2	Intent IDs	Policy, FirewallRule
A3	Policy→Config	420	\$5.1.2	Neo4j context	Configuration
A4	Planning	243	\$5.1.2	Changeset+configs	MigrationPlan
A5	Orchestration	350	\$5.1.3	Configuration	DeploymentEvent
A6	Monitoring	380	\$5.2.1	Device list	Telemetry, OpState
A7	Assessment	417	\$5.2.2	POR vs As-Built	Assessment, Incident
A8	Action	405	\$5.2.3	Incident	Remediation
A9	Abstraction	—	\$5.2	Assessments	Aggregated scores
A10	Reporting	—	\$5.2	Aggregations	Reports

Dispatch-table architecture. A recurring pattern across agents is the use of dispatch tables to decouple vendor- or type-specific logic from the agent’s control flow. Three dispatch tables are central:

- (1) **_KIND_MAP** (Provisioner): maps (vendor, platform) to containerlab kind, image, and type. Adding Nokia SR Linux, FRR, and VyOS required one entry each—three lines total.
- (2) **_TEMPLATE_CHAINS** (Agent 3): maps platform to an ordered list of Jinja2 template filenames. Adding FRR required one entry (three templates); adding SR Linux required one entry (three templates).
- (3) **_VENDOR_EXECUTORS** (Agent 5): maps platform to a callable that pushes configuration. Adding the gNMI transport for SR Linux required one new callable and a two-line dispatch override keyed on IBN_SRLINUX_TRANSPORT.

This architecture-as-data pattern means that adding a fourth vendor (e.g. Arista cEOS) requires: one **_KIND_MAP** entry, one **_TEMPLATE_CHAINS** entry (pointing to new Jinja2 templates), one executor function, and a Dockerfile for the container image. No existing agent code changes.

The detailed operation of each agent is described in Section 2.2. Here we note one additional design decision per agent that is not covered by the pipeline description:

- A1:** Interactive dialogue support. When a population VLAN is ambiguous (e.g. multiple valid access-list templates apply), Agent 1 can invoke a `dialogue_fn` callback with a prompt and a list of choices (**_ACCESS_TEMPLATES**: four templates labelled a/b/c/d). In batch mode the dialogue is auto-resolved to the first template; in interactive mode the operator selects.
- A2:** Zone inference is rule-based, not LLM-based. The mapping from target string to source zone (DMZ → DMZ_INFRA, guest → GUEST, else USER) is three lines of code. This is deliberately minimal: the HLD’s zone assignment table is authoritative, and the agent’s role is to decompose, not to infer.
- A3:** Content-hash-based change detection. Each rendered configuration is SHA-256 hashed (truncated to 16 hex chars). If the hash matches the previous configuration for the same device, the render is considered a no-op and no new Configuration node is written. This prevents redundant deployment events when the HLD is committed with cosmetic changes that do not affect device state.
- A4:** Supersession logic. If a PENDING migration plan already exists when a new commit is processed, the old plan is marked

SUPERSEDED rather than deleted. The superseded plan stays in the graph for audit, linked to the new plan via a PRECEDED_BY edge.

- A7:** Decorator-based drift detector registration. New drift detectors are added by writing a function decorated with `@drift_detector("name")`; the decorator registers it in a global **_DRIFT_DETECTORS** dict. The assessment loop iterates over all registered detectors. Three are currently registered: `firewall_rule`, `interface_state`, `vrrp_role`.
- A8:** Severity classification is intentionally decoupled from Agent 7’s. Agent 7 classifies assessment severity (how bad is the drift?); Agent 8 classifies action severity (what should we do about it?). The two classifications use different inputs: A7 uses drift-event count and compliance percentage; A8 uses business-impact signals (service impact, redundancy status, device count).

5 THREE-TIER MEMORY

5.1 Architecture Overview

The three-tier memory architecture separates three qualitatively different kinds of knowledge:

- Tier 1: Neo4j SSoT (*what is*).** Structured, typed, queryable network state across nine layers. Accessed via Bolt (port 7687). Every agent reads and writes this tier on every cycle. Latency: ~200 ms per Cypher query.
- Tier 2: Live-Memory (*why it is so*).** Working memory for agent reasoning. Exposed via Model Context Protocol (MCP) Streamable HTTP on port 8002, backed by S3 object storage. Each model state has a dedicated space (e.g. `ibn-candidate-001`, `ibn-por-v1`, `ibn-deploy-evt1`, `ibn-asbuilt-snap1`) plus two loop-scoped spaces (`ibn-loop-inner`, `ibn-loop-outer`). Agents write structured notes via `live_note()` calls that include: agent ID, timestamp, operation type, inputs summary, outputs summary, and free-text rationale. The notes accumulate in a *bank* per space; the bank is periodically consolidated.
- Tier 3: Graph-Memory (*what was learned*).** Long-term knowledge graph with vector embeddings. Exposed via MCP on the same port (namespaced as `IBN_LIFECYCLE_*` labels in a separate Neo4j instance) plus Qdrant for BGE-M3 1024-dimension embeddings. This tier stores entities and relations extracted from consolidated bank files using an LLM guided by the IBN Lifecycle Ontology. Agents query this tier via

`question_answer()` for historical precedent-based decision making (graph-guided RAG).

5.2 The IBN Lifecycle Ontology

The Graph-Memory entity extraction is guided by a domain-specific ontology (`ibn_lifecycle_ontology.yaml`, 472 lines) that defines 14 entity classes across four families and 8 relationship types.

Entity families.

- (1) **Decisions & Rationale** (5 classes): `PolicyDecision` (firewall/zone decisions), `DesignChoice` (architectural decisions), `TradeOff` (competing approaches evaluated), `RejectedAlternative` (explicitly rejected options), `ApprovalRecord` (human sign-offs with operator identity and timestamp).
- (2) **Incidents & Remediation** (5 classes): `DriftEvent` (POR vs. As-Built mismatches), `RootCauseAnalysis` (causal chains for drift), `RemediationAction` (auto-fix or rollback records), `EscalationRecord` (human escalation events), `ComplianceTrend` (compliance trajectory across iterations: improving, stable, degrading).
- (3) **Operations & Deployment** (4 classes): `DeploymentNarrative` (config push records with per-device outcome), `ConfigurationRationale` (why a template rendered the way it did—e.g. “VLAN 100 assigned to subinterface 100 because the HLD population table maps Employee to VLAN 100”), `RollbackRecord` (rollback events with before/after config hashes), `VerificationResult` (post-push verification outcomes).
- (4) **Knowledge & Patterns** (4 classes): `RecurringPattern` (drift/latency/error correlations observed across multiple cycles), `LessonLearned` (operational guidance distilled from incident history), `CapacityInsight` (resource constraints—e.g. “host memory gates provisioning beyond 12 nodes”), `CorrelationFinding` (metric correlations discovered by the aggregation pipeline).

Relationship types. Eight typed relationships connect entities across families:

- (1) `CAUSED_BY`: drift/remediation to root cause.
- (2) `RESOLVED_BY`: incident to fix action.
- (3) `PRECEDED`: temporal ordering of events.
- (4) `JUSTIFIED_BY`: decision to supporting rationale.
- (5) `CONTRADICTS`: new finding vs. old lesson/pattern.
- (6) `CONFIRMS`: event that validates a known pattern.
- (7) `SIMILAR_TO`: cross-iteration pattern matching (primary RAG retrieval link).
- (8) `INFORMED_BY`: decision informed by historical precedent (explicit query provenance).

Extraction hints. The ontology includes natural-language extraction hints (lines 432–472) that guide the LLM’s entity recognition. For example, the `DriftEvent` class is triggered by phrases like “drift detected”, “non-compliant”, “missing rule”; the `LessonLearned` class by “lesson:”, “best practice”, “should always”. These hints are injected into the LLM’s system prompt during `graph_push()` to improve extraction precision without fine-tuning.

5.3 Consolidation Pipeline

The `ConsolidationManager` (395 lines) orchestrates the Tier 2→Tier 3 data flow:

- (1) **Trigger.** Consolidation fires on three events: (a) a model-state transition (e.g. `CANDIDATE`→`POR` after approval), (b) the inner loop completing a full cycle, or (c) the note count in a Live-Memory space exceeding a configurable threshold (default: 20 notes).
- (2) **Consolidate.** The manager calls `bank Consolidate(space_id)` on the Live-Memory MCP server, which invokes an LLM to summarise the accumulated notes into a structured bank file. The LLM is configured via `LLMAAS_API_URL` and `LLMAAS_MODEL`; the current testbed defaults to Claude Haiku 4.5 via Cloud-Temple’s LLMAAS proxy, with Qwen3-235B as an alternative. The consolidation is space-specific: each model-state space has its own consolidation cadence and rules.
- (3) **Push.** If the transition is *push-eligible* (to `POR`, `DEPLOYED`, or `AS_BUILT`), the manager reads all bank files via `bank_read_all(space_id)` and pushes them to Graph-Memory via `graph_push_batch(memory, bank_files, space_id)`. The Graph-Memory server base64-encodes each bank file, invokes the LLM with the IBN Lifecycle Ontology as context, extracts typed entities and relations, embeds them with BGE-M3 (1024 dimensions), and stores them in Qdrant + the Graph-Memory Neo4j instance (namespaced under `IBN_LIFECYCLE_*` labels).
- (4) **Query.** Any agent can query Tier 3 via `question_answer(memory, question, max_results=5)`, which performs graph-guided RAG: the question is embedded, nearest-neighbour entities are retrieved from Qdrant, their graph neighbourhood is traversed via `SIMILAR_TO` and `INFORMED_BY` edges, and the combined context is returned to the caller.

The consolidation pipeline is asynchronous and non-blocking: it runs after the pipeline cycle completes and never delays the fulfilment or assurance path. The manager also runs a background sweep loop (default interval: 3,600 s) that checks all spaces for threshold-exceeding note counts, providing a safety net for spaces that miss event-driven triggers.

5.4 LLM Backend Configuration

The LLM is used in exactly two places: Live-Memory consolidation (bank summarisation) and Graph-Memory ingestion (entity/relation extraction). Neither is in the request path of any pipeline stage.

The backend is configurable at deploy time via environment variables:

Listing 5. LLM configuration in `docker-compose.yml`.

```
LLMAAS_API_URL=http://host.docker.internal:8004/v1
LLMAAS_API_KEY=${LLMAAS_API_KEY}
LLMAAS_MODEL=${LLMAAS_MODEL:-claude-haiku-4-5-20251001}
LLMAAS_EMBEDDING_MODEL=all-MiniLM-L6-v2
LLMAAS_EMBEDDING_DIMENSIONS=1024
```

Swapping or disabling the LLM backend degrades long-term memory quality (fewer entities extracted, coarser summaries) but

does not affect the pipeline’s ability to parse, render, push, and verify. The deterministic pipeline and the LLM-assisted memory are independently testable and independently deployable.

6 IMPLEMENTATION

6.1 Testbed

The prototype runs on a single 12 GB ARM64 virtual machine (Ubuntu 24.04, Linux 7.0.0, Python 3.14) with Docker and Containerlab [1]. The lab topology (clab-ibnlab-switches.yml) defines 16 containers:

- 3 Nokia SR Linux (ghcr.io/nokia/srlinux:latest, kind nokia_srlinux, IXR-D2L profile): acc-sw-01 (pioneer from Slice 2), acc-01, agg-01.
- 2 FRRouting (ibn-frr:local, custom Debian-slim image, 62 MB, multi-arch): edge-01, edge-02.
- 4 VyOS firewalls (ghcr.io/sysoleg/vyos-container:latest): usf-01, usf-02, dmzfw-01, dmzfw-02.
- 7 host stubs (lightweight Linux containers simulating end hosts in each population VLAN).

The management network is 192.168.100.0/24. SR Linux containers expose gNMI on port 57400 and NETCONF on port 830 by default. FRR containers run zebra, bgpd, ospfd, ospf6d, and isisd daemons with tini as PID 1. The FRR image is built from a committed Dockerfile (infra/frr-image/Dockerfile) to work around the upstream frouting/frr image being amd64-only.

Infrastructure services (Neo4j 5.x, Live-Memory MCP, Graph-Memory MCP, Qdrant, Redis) run via docker-compose.yml on the same host. Total memory usage with all 16 containers plus infrastructure: ~10 GB, leaving ~1.9 GB headroom.

6.2 Incremental Delivery: Six Slices

The implementation was delivered in six incremental slices, each building on the previous and each validated with unit tests and a live end-to-end run. Table 5 summarises the slices.

Each slice was committed with a plan document (PLAN_Slice<N>.md) before implementation and a fix-notes document (FIX_NOTES_Slice<N>.md) after, capturing bugs found, fixes applied, architectural decisions made during execution, and known limitations. This convention was adopted after Slice 2 and applied retroactively; it provides a readable post-mortem record that survives outside the development conversation log.

6.3 Failure Modes and Fixes

Several non-trivial failures were encountered and resolved during implementation:

MCP hang on Python 3.14. The `asyncio.run()` call hung in teardown after the MCP `streamablehttp_client` context manager exited, due to an anyio TaskGroup orphan-task bug. After evaluating factory patterns, `asyncio.wait_for`, and `ThreadPoolExecutor` (all of which also hung), the fix was a persistent background event loop in a daemon thread with `asyncio.run_coroutine_threadsafe()`. This pattern is now used by both `LiveMemoryClient` and `GraphMemoryClient`.

Batch provisioning timeout. Per-device `clab deploy --reconfigure` re-processes the entire topology on each invocation. With four new devices, the third sequential deploy exceeded the 300 s timeout. The fix (`provision_batch()`) performs a single YAML write and a single deploy for the entire batch, completing in ~10 s.

FRR ARM64 mismatch. The upstream `frouting/frr:latest` image is amd64-only and crashes under QEMU on the ARM64 host. The fix was a custom `ibn-frr:local` image built from `debian:stable-slim + apt install frr` (62 MB, native ARM64).

Neo4j OOM under memory pressure. On the original 8 GB host, Neo4j was OOM-killed (exit 137) when the fifth SR Linux container came up alongside four VyOS firewalls. The host was resized to 12 GB, which provides ~4 GB free after all containers and infrastructure.

Graph-Memory MCP schema mismatch. The `GraphMemoryClient` initially sent field names that did not match the MCP server’s expected schema (name instead of `memory_id`, content instead of `content_base64`). The `LLMAAS_API_URL` was also missing the `/v1` path prefix. Both were fixed by reading the server’s OpenAPI spec and aligning field names.

6.4 Measurements

7 DISCUSSION AND RELATED WORK

7.1 Comparison with Vendor IBN Suites

Cisco DNA Center, Juniper Apstra, and Arista CloudVision all implement forms of closed-loop intent, but they share three properties our system avoids:

- (1) **Vendor lock-in.** Each system manages only its own vendor’s devices. Our dispatch-table architecture supports three vendors (VyOS, SR Linux, FRR) with a fourth (Arista cEOS) requiring only a new dispatch entry and template chain.
- (2) **Proprietary intent model.** Intent is authored in a vendor-specific GUI or API; it cannot be diffed, branched, or code-reviewed with standard tooling. Our intent is a Markdown file under git.
- (3) **Opaque state.** The internal state model is not directly queryable by operators. Our Neo4j SSOT with its nine-layer ontology is fully queryable via Cypher, and every agent write is dual-sinked to Live-Memory for rationale capture.

7.2 Cloud-Temple’s Live-Memory and Graph-Memory

Tiers 2 and 3 of our memory architecture are built on two open-source projects developed by Cloud-Temple (Apache 2.0 licence): **Live-Memory** [6] and **Graph-Memory** [5]. Both are exposed as MCP (Model Context Protocol) Streamable HTTP servers and were designed for multi-agent LLM collaboration, following the framework described by Tran et al. [10].

Live-Memory provides a dual-mode working memory system: a *Live* mode for real-time, append-only note capture (the `live_`

Table 5. Six implementation slices: scope, key deliverables, and test counts.

Slice	Name	Key deliverable	Tests
1	HLD Pipeline	HLD parser, A1 ingestion, A2 decomposition, A3 rendering (VyOS only), A4 planning, A5 push via SSH, git hook trigger	52
2	Multi-vendor	SR Linux template chain (srl_base, srl_interfaces, srl_vlans), docker exec sr_cli executor, pioneer switch acc-sw-01 push	72
3	Provisioning	Containerlab YAML mutation, provision() and retire(), _KIND_MAP dispatch, lifecycle events	86
4	Approval gate	A4 severity classifier, MigrationPlan node, pending-plan Markdown, approve/reject CLI tool, resume_from_approval(), auto-approve for LOW/INFO	100
5	FRR + batch	FRR as third vendor (Dockerfile, 3 templates, vtysh executor), provision_batch() fix, 4 new devices provisioned, VM resize to 12 GB	118
7a	gNMI transport	pygnmi executor for SR Linux, CLI→YANG translator (10 rules), IBN_SRLINUX_TRANSPORT env flag, live SetRequest + verification	140

Table 6. Pipeline timing for representative operations.

Operation	Wall-clock time
No-op HLD commit (full cycle)	8–10 s
VLAN addition (1 device affected)	10–12 s
Device addition (4 devices, batch)	20–40 s
gNMI SetRequest (7 updates)	<1 s
sr_cli push (typical)	~2 s
vtysh push (typical)	~1.5 s
SSH push to VyOS	~3 s
Neo4j Cypher query (avg of 10 A3 queries)	~200 ms

note() calls used by every agent), and a *Bank* mode for LLM-consolidated structured documentation. Notes accumulate in per-space banks backed by S3 object storage; when a consolidation trigger fires (model-state transition, note-count threshold, or inner-loop completion), the server invokes an LLM to summarise the accumulated notes into a structured Markdown bank file. The bank files are the input to Tier 3. Live-Memory’s space model maps naturally onto our five-state lifecycle: each model state gets a dedicated space (ibn-candidate-*, ibn-por-*, etc.), plus two loop-scoped spaces (ibn-loop-inner, ibn-loop-outer). This separation ensures that consolidation is scoped to the phase that produced the notes, avoiding cross-contamination between, say, planning-phase reasoning and deployment-phase observations.

Graph-Memory implements a “Knowledge Graph as a Service” pattern: ontology-driven LLM extraction of typed entities and relationships from natural-language documents, stored in a namespace-isolated Neo4j graph (IBN_LIFECYCLE_* labels) with BGE-M3 1024-dimension vector embeddings in Qdrant. Three capabilities are central to our architecture:

- **Ontology-driven extraction.** Graph-Memory accepts a YAML-defined ontology that constrains the entity and relationship types the LLM can produce during ingestion. Our IBN Lifecycle Ontology (472 lines, 14 entity classes, 8 relationship types — see Section 5.2) guides extraction so that, for example, a note mentioning “drift detected on VLAN 110” produces a typed `DriftEvent` entity linked via `CAUSED_BY` to the relevant `RootCauseAnalysis`, rather than a generic text chunk.
- **Namespace isolation.** Each Graph-Memory “memory” gets prefixed Neo4j labels, ensuring complete isolation from the

SSoT’s nine-layer ontology operating in the same Neo4j instance. The SSoT graph and the knowledge graph coexist without schema interference.

- **Graph-guided RAG.** When an agent queries Graph-Memory via `question_answer()`, the system first identifies relevant entities through fuzzy and semantic search in Neo4j, then constrains the Qdrant vector search to only those source documents. This produces precise, context-filtered answers rather than the noisy results typical of pure vector RAG.

Cloud-Temple also provides the **LLMaaS** (LLM as a Service) API endpoint used by both Live-Memory and Graph-Memory for consolidation and entity extraction. The current testbed defaults to Claude Haiku 4.5 via Cloud-Temple’s LLMaaS proxy (api.ai.cloud-temple.com), with Qwen3-235B as an alternative hosted on the same infrastructure. The LLMaaS endpoint is OpenAI-API-compatible, so swapping models or providers requires only changing the `LLMAAS_MODEL` environment variable.

The contribution of Live-Memory and Graph-Memory to this work is structural: they provide the working-memory and long-term knowledge tiers that, together with Neo4j, realise the three-tier separation of *what is*, *why it is so*, and *what was learned*. Without these tiers, the pipeline would still render and push configuration, but the operational rationale behind each agent decision would be lost, and the system could not answer historical queries like “what happened last time we saw VLAN 110 latency spikes?” with typed, graph-grounded answers.

7.3 Relationship to MALT

Google’s MALT [9] introduced the concept of modelling network topologies at multiple levels of abstraction, driven by the observation that no single abstraction level serves all consumers. Our nine-layer ontology is directly inspired by MALT but differs in scope: MALT focuses on topology abstraction for traffic engineering in a homogeneous datacentre; our ontology spans the full IBN lifecycle (from business intent through deployment to compliance) in a heterogeneous multi-vendor campus. The five concurrent model states extend MALT’s abstraction-level concept with temporal semantics: the same intent exists simultaneously as `CANDIDATE`, `POR`, `DEPLOYED`, and `AS_BUILT`, each reflecting a different point in the fulfilment lifecycle.

7.4 GitOps for Networks

The GitOps pattern—declarative desired state in a Git repository, reconciled by an automated operator—is well-established for Kubernetes (Flux, ArgoCD) but less explored for network infrastructure. Network-to-Code’s Nautobot and Netbox provide source-of-truth databases but do not close the loop: they record intent but do not render, push, or verify. Our system closes the loop end-to-end: the HLD commit triggers rendering, the approval gate provides human oversight, and the assurance loop detects drift and auto-remediates.

7.5 YANG and NETCONF/gNMI

The move from CLI-based config push to schema-validated gNMI (Slice 7a) aligns with the broader industry trend toward model-driven management [7]. Our experience confirms that gNMI’s structured SetRequest semantics (atomic commit, typed paths, JSON-IETF encoding) are a better fit for programmatic push than piping CLI commands through `docker exec`. The narrow CLI→YANG translator (`_srlinux_yang.py`) is a pragmatic compromise: it keeps the A3 template chain transport-agnostic (the same `set / commands` feed both the CLI and gNMI paths) while the translator absorbs the encoding difference. A future slice (7c) will render OpenConfig JSON directly, eliminating the translation step.

7.6 Lessons Learned

- (1) **The HLD-as-intent commitment is viable but demanding.** The HLD must be structured enough for a deterministic parser to extract typed records. In practice this means the Device Inventory Table and VLAN assignment tables must follow a fixed Markdown table format. Prose sections (design rationale, capacity planning) are not parsed but are preserved in Live-Memory notes for future LLM-driven extraction.
- (2) **Batch provisioning matters.** The per-device `clab deploy --reconfigure` path scales quadratically with topology size because each invocation re-processes the entire YAML. The batch path (one write, one deploy) reduces this to constant time for the deploy step.
- (3) **Three tiers are the right number.** Early designs considered a two-tier model (Neo4j + LLM-backed knowledge). The working-memory tier (Live-Memory) turned out to be essential: agents need a fast, low-latency scratchpad for intra-cycle communication that is richer than Neo4j properties but cheaper than a full knowledge-graph ingestion.
- (4) **LLMs belong in the memory tier, not the pipeline.** Every attempt to introduce LLM inference into the pipeline (e.g. LLM-based policy decomposition in A2, LLM-based template selection in A3) degraded cycle time by 10–30× and introduced non-determinism that broke idempotency invariants. The current separation—deterministic pipeline, LLM-assisted memory—is a deliberate architectural choice, not a limitation.
- (5) **Dispatch tables scale better than inheritance.** The vendor-extensibility pattern (one table entry per vendor)

has proven more maintainable than a class-hierarchy approach. Adding FRR as a third vendor touched exactly one line per dispatch table plus a Dockerfile and three Jinja2 templates; no existing code was modified.

8 CONCLUSION

We have presented an Intent-Based Networking closed-loop control system that treats a conventional Markdown High-Level Design document as the sole source of intent, triggered by `git commit` and reconciled to live network state through a pipeline of ten specialised agents grounded in RFC 9315. The system is backed by a Neo4j knowledge graph with a nine-layer ontology and five concurrent model states, a three-tier memory architecture that cleanly separates structured state, working memory, and long-term knowledge, and a dispatch-table-driven multi-vendor rendering and push pipeline supporting VyOS, Nokia SR Linux (via both CLI and gNMI), and FRRouting.

The implementation was delivered incrementally across six slices on a live Containerlab testbed with 16 nodes, validated by 140 unit tests with zero cross-slice regressions, and operates with a full-cycle time of 8–10 seconds. The core pipeline is entirely deterministic and LLM-free; large language models are used exclusively for memory consolidation and knowledge extraction, preserving the system’s correctness properties while enabling rich operational knowledge capture.

The key architectural insight is that the gap between an HLD and a running network is not a translation problem that requires a proprietary controller—it is a reconciliation problem that can be decomposed into narrow, testable agents operating over a shared graph, triggered by the same version-control tooling that developers already use. The HLD is not a specification of intent that must be re-encoded into a machine format; it *is* the intent, and the closed loop’s job is to make reality match it.

Future work. Immediate next steps include: NETCONF-over-SSH transport for SR Linux (Slice 7b), OpenConfig shared templates across vendors (Slice 7c), gNMI streaming telemetry for sub-second inner-loop monitoring (Slice 7d), and full implementation of Agents 9 and 10 for abstraction and human-facing reporting. Longer-term, we plan to evaluate predictive drift detection using Graph-Memory’s historical knowledge base and to extend the testbed to include Arista cEOS and Juniper cRPD.

REFERENCES

- [1] [n. d.]. Containerlab: Container-based Networking Labs. <https://containerlab.dev/>. ([n. d.]). Accessed 2026-04-15.
- [2] [n. d.]. FRRouting Project. <https://frrouting.org/>. ([n. d.]). Accessed 2026-04-15.
- [3] [n. d.]. Neo4j Graph Database. <https://neo4j.com/>. ([n. d.]). Accessed 2026-04-15.
- [4] A. Clemm, L. Ciavaglia, L. Z. Granville, and J. Tantsura. 2022. Intent-Based Networking – Concepts and Definitions. RFC 9315, Internet Engineering Task Force. (Oct. 2022). <https://doi.org/10.17487/RFC9315>
- [5] Cloud-Temple. 2025. Graph-Memory: Knowledge Graph as a Service MCP Server for AI Agents. <https://github.com/Cloud-Temple/graph-memory>. (2025). Apache 2.0 licence. Accessed 2026-04-15.
- [6] Cloud-Temple. 2025. Live-Memory: Working Memory MCP Server for Collaborative AI Agents. <https://github.com/Cloud-Temple/live-memory>. (2025). Apache 2.0 licence. Accessed 2026-04-15.
- [7] OpenConfig Working Group. [n. d.]. gNMI: gRPC Network Management Interface. <https://github.com/openconfig/gnmi>. ([n. d.]). Accessed 2026-04-15.
- [8] Anton Karneliuk. [n. d.]. pygnmi: Pure Python gNMI client. <https://github.com/akarneliuk/pygnmi>. ([n. d.]). Accessed 2026-04-15.

- [9] Jeffrey C. Mogul, Drago Crnković, John Evans, Jakub Łopuszyński, Amin Vahdat, and Ryan Woodard. 2020. Experiences with Modeling Network Topologies at Multiple Levels of Abstraction. In *Proc. USENIX NSDI '20*.
- [10] Thanh Tran et al. 2025. Multi-Agent LLM Systems: Techniques, Challenges and Future Directions. *arXiv preprint arXiv:2501.06322* (2025).